

Subprograme în Python — Rezolvări și Teorie

Clasa a IX-a · Profil Matematică-Informatică · Intensiv Informatică

Document destinat profesorului — conține rezolvările complete și note metodologice.

Teorie completă — Subprograme în Python

1. Ce este un subprogram?

Un **subprogram** (funcție) este un bloc de instrucțiuni care îndeplinește o sarcină bine definită, primește date de intrare (parametri) și poate returna un rezultat. Scopul principal este **reutilizarea codului** și **proiectarea modulară**.

2. Structura unui subprogram în Python

```
def nume_functie(param1, param2, ...):  
    # corpul funcției — instrucțiunile  
    # calcule, decizii, bucle etc.  
    return rezultat # opțional  
  
# Apel:  
variabila = nume_functie(val1, val2)
```

PARAMETRI FORMALI VS EFECTIVI

Formali — numiți în definiție: `def f(a, b)`

Efectivi — valorile la apel: `f(3, 7)`

La apel, Python asociază: `a = 3, b = 7`

VARIABLE LOCALE VS GLOBALE

Locale — declarate în funcție; dispar după apel.

Globale — declarate în exterior; vizibile în tot programul (accesate cu `global x` dacă se modifică).

RETURN

Oprește execuția funcției și trimite valoarea.

Fără `return` → funcția returnează `None`.

Se pot returna mai multe valori: `return a, b` → tuplu.

FUNCȚII PREDEFINITE UTILE

`abs(x)` — valoare absolută

`round(x, n)` — rotunjire la n zecimale

`int(x)`, `float(x)`, `str(x)` — conversii

`len(l)`, `min(l)`, `max(l)`, `sum(l)`

`math.sqrt(x)` — radical (import math)

3. Proiectare modulară

O problemă complexă se descompune în subprobleme. Fiecare subproblemă devine un subprogram. Subprogramele se pot apela reciproc. Avantaje: cod mai clar, ușor de testat, reutilizabil.

SIMPLU

EXERCIȚIUL 1 (5P)

Scrie un subprogram `salut` care primește un parametru `nume` și afișează „*Bună ziua, [nume]!*”. Apelează cu propriul prenume.

REZOLVARE

```
def salut(nume):  
    print(f"Bună ziua, {nume}!")  
  
salut("Maria")
```

OUTPUT

Bună ziua, Maria!

Concepte cheie: funcția nu returnează nimic — scopul ei este să afișeze. Parametrul `nume` primește valoarea `"Maria"` la apel. Se folosește un f-string pentru inserarea variabilei în text.

Sfat metodologic: Elevii trebuie să facă diferența dintre `print()` în interiorul funcției și `return + print()` în exterior. Aceasta clarifică confuzia frecventă afișare ≠ returnare.

EXERCIȚIUL 2 (5P)

Definește un subprogram `patrat` care primește `n` și returnează pătratul său. Afișează `patrat(7)`.

REZOLVARE

```
def patrat(n):  
    return n ** 2  
  
print(patrat(7))    # 49
```

OUTPUT

49

EXERCIȚIUL 3 (5P)

Subprogram `media` cu trei numere reale — returnează media aritmetică. Citire de la tastatură.

REZOLVARE

```
def media(a, b, c):  
    return (a + b + c) / 3  
  
a = float(input("a = "))  
b = float(input("b = "))  
c = float(input("c = "))  
print(f"Media: {media(a, b, c):.2f}")
```

De reținut: `input()` returnează întotdeauna un șir de caractere, deci e necesar să facem conversia cu `float()`.

EXERCIȚIUL 4 (5P)

Completează `este_par`. Ce afișează programul?

```
def este_par(n):  
    _____  
  
print(este_par(4))  
print(este_par(7))
```

REZOLVARE

```
def este_par(n):  
    return n % 2 == 0  
  
print(este_par(4))    # True  
print(este_par(7))    # False
```

OUTPUT

True False

Expresia `n % 2 == 0` este ea însăși o expresie booleană — returnează direct `True` sau `False`, fără a fi nevoie de `if/else`.

MEDIU

EXERCIȚIUL 5 (10P)

Subprogram `distanța(x1, y1, x2, y2)` — distanța euclidiană dintre două puncte. Testează cu A(0,0) și B(3,4).

REZOLVARE

```
import math

def distanța(x1, y1, x2, y2):
    return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

d = distanța(0, 0, 3, 4)
print(f"Distanța A-B: {d:.2f}")    # 5.00
```

OUTPUT

Distanța A-B: 5.00

Formula: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ — teorema lui Pitagora generalizată. Verificare: $\sqrt{9+16} = \sqrt{25} = 5$. Modulul `math` trebuie importat; alternativ se poate folosi `** 0.5` în loc de `sqrt`.

EXERCIȚIUL 6 (10P)

Subprogram `maxim_lista(lst)` — valoarea maximă fără `max()`. Testează cu `[3, 17, 8, 42, 5]`.

REZOLVARE

```
def maxim_lista(lst):
    if len(lst) == 0:
        return None
    maxim = lst[0]          # pornim cu primul element
    for element in lst:
        if element > maxim:
            maxim = element
    return maxim

numere = [3, 17, 8, 42, 5]
print(maxim_lista(numere))    # 42
```

OUTPUT

42

Algoritm de parcurgere liniară cu reținerea maximului: se inițializează maximul cu primul element, apoi se parcurge lista și se actualizează dacă se găsește un element mai mare. Verificăm cazul special al listei vide.

EXERCIȚIUL 7 (10P)

Subprogram `numar_vocale(sir)` — numără vocalele dintr-un șir. Testează cu „Informatica”.

REZOLVARE

```
def numar_vocale(sir):
    vocale = "aeiouAEIOU"
    contor = 0
    for caracter in sir:
        if caracter in vocale:
            contor += 1
    return contor

print(numar_vocale("Informatica"))    # 5
```

OUTPUT

5

Vocalele din „Informatica”: I, o, a, i, a → 5. Operatorul `in` verifică apartenența unui caracter la un șir.

Variantă mai concisă (pentru elevi avansați):

```
return sum(1 for c in sir if c in "aeiouAEIOU")
```

EXERCIȚIUL 8 (10P)

Identifică cele două erori din codul dat și rescrie versiunea corectă.

```
def suma_ptrate(a, b)
    rezultat = a^2 + b^2
    return rezultat

print(suma_pttrate(3, 4))
```

REZOLVARE — IDENTIFICAREA ERORILOR

Eroarea 1: Lipsește `:` la sfârșitul liniei `def suma_pttrate(a, b)` → eroare de sintaxă (SyntaxError).

Eroarea 2: Operatorul `^` în Python nu înseamnă ridicare la putere — este operatorul XOR pe biți. Rezultatul ar fi `3^2 = 1` și `4^2 = 6`, deci `1+6=7` în loc de `9+16=25`. Ridicarea la putere se face cu `**`.

```
# Cod corectat:
def suma_pttrate(a, b):          # adăugat ":"
    rezultat = a ** 2 + b ** 2  # "^" înlocuit cu "**"
    return rezultat

print(suma_pttrate(3, 4))      # 25
```

OUTPUT CORECT

25

AVANSAT

EXERCIȚIUL 9 (15P)

Trei subprograme: `cmmdc`, `cmmnc` (folosește `cmmdc`), `simplifica` (folosește `cmmdc`). Testează: `simplifica(12, 18) → (2, 3)`.

REZOLVARE

```
def cmmdc(a, b):
    # Algoritmul lui Euclid prin împărțiri repetate
    while b != 0:
        a, b = b, a % b
    return a

def cmmnc(a, b):
    # cmmnc(a,b) = a * b / cmmdc(a,b)
    return a * b // cmmdc(a, b)

def simplifica(numarator, numitor):
    d = cmmdc(numarator, numitor)
    return (numarator // d, numitor // d)

# Teste
print(simplifica(12, 18))    # (2, 3)
print(cmmnc(4, 6))          # 12
print(cmmdc(48, 18))        # 6
```

OUTPUT

(2, 3) 12 6

Algoritmul lui Euclid: se înlocuiește repetat (a, b) cu (b, a mod b) până când b = 0; atunci a este CMMDC.

Proiectare modulară: `cmmnc` și `simplifica` se bazează pe `cmmdc` — fiecare funcție face un singur lucru bine definit. Aceasta este esența modularității.

EXERCITIUL 10 (15P)

Aplicație modulară pentru catalog: `adauga_nota`, `calculeaza_medie`, `afiseaza_raport`. Citire 5 note.

REZOLVARE

```
def adauga_nota(lista, nota):
    if 1 <= nota <= 10:
        lista.append(nota)
        return True
    print(f"Nota {nota} este invalidă (trebuie 1-10).")
    return False

def calculeaza_medie(lista):
    if len(lista) == 0:
        return None
    return sum(lista) / len(lista)

def afiseaza_raport(lista):
    if len(lista) == 0:
        print("Nicio notă înregistrată.")
        return
    print(f"Note: {lista}")
    print(f"Media: {calculeaza_medie(lista):.2f}")
    print(f"Minimul: {min(lista)} | Maximul: {max(lista)}")

# Program principal
note = []
for i in range(5):
    n = float(input(f"Nota {i+1}: "))
    adauga_nota(note, n)

afiseaza_raport(note)
```

EXEMPLU DE OUTPUT (NOTE: 8, 9, 7, 10, 6)

```
Note: [8.0, 9.0, 7.0, 10.0, 6.0] Media: 8.00 Minimul: 6.0 | Maximul: 10.0
```

Fiecare funcție are o responsabilitate clară: `adauga_nota` validează, `calculeaza_medie` calculează, `afiseaza_raport` prezintă. Funcțiile returnează valori utile (`True/False`, `None`), permițând tratarea erorilor în programul principal.

EXERCIȚIUL 11 (15P)

`este_prim(n) → prime_din_lista(lst)`. Testează cu `[2, 3, 4, 9, 11, 15, 17, 20]`.

REZOLVARE

```
import math

def este_prim(n):
    if n < 2:
        return False
    for d in range(2, int(math.sqrt(n)) + 1):
        if n % d == 0:
            return False
    return True

def prime_din_lista(lst):
    rezultat = []
    for element in lst:
        if este_prim(element):
            rezultat.append(element)
    return rezultat

numere = [2, 3, 4, 9, 11, 15, 17, 20]
print(prime_din_lista(numere))    # [2, 3, 11, 17]
```

OUTPUT

[2, 3, 11, 17]

Optimizare cheie: parcurgem divizorii doar până la $\sqrt{n}$, nu până la n — suficient pentru a verifica primalitatea.

Modularitate: `prime_din_lista` deleghează testul de primalitate către `este_prim` — nu duplică logica.

Variantă cu list comprehension (avansați):

```
return [x for x in lst if este_prim(x)]
```

EXERCITIUL 12 (15P)

Firmă de livrări: cost = 5 lei/kg; reducere 10% dacă > 10 kg; taxă 20 lei dacă > 100 km. Proiectare modulară cu min. 3 subprograme.
Test: 12 kg, 150 km.

REZOLVARE — PROIECTARE MODULARĂ

```
# Subprogram 1: calculul costului de bază
def cost_baza(greutate):
    return greutate * 5

# Subprogram 2: aplicarea reducerii
def aplica_reducere(cost, greutate):
    if greutate > 10:
        return cost * 0.9    # reducere 10%
    return cost

# Subprogram 3: aplicarea taxei de distanță
def taxa_distanta(cost, distanta):
    if distanta > 100:
        return cost + 20
    return cost

# Subprogram 4 (coordonator): calculul total
def calculeaza_cost_total(greutate, distanta):
    cost = cost_baza(greutate)
    cost = aplica_reducere(cost, greutate)
    cost = taxa_distanta(cost, distanta)
    return cost

# Program principal
g = float(input("Greutate (kg): "))
d = float(input("Distanță (km): "))
total = calculeaza_cost_total(g, d)
print(f"Cost total: {total:.2f} lei")
```

OUTPUT PENTRU 12 KG, 150 KM

Cost total: 74.00 lei

Calcul pas cu pas: cost bază = $12 \times 5 = 60$ lei → cu reducere 10% = 54 lei → cu taxă distanță (+20) = 74 lei.

Principiu cheie de proiectare modulară: fiecare subprogram face un singur lucru. `calculeaza_cost_total` este subprogramul coordonator — apelează celelalte subprograme fără să conțină logica lor. Aceasta face codul ușor de modificat: dacă procentul reducerii se schimbă, modificăm doar `aplica_reducere`.